

Artificial Intelligence — Lecture 5

Adversarial Search and Game Playing

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 5 at a Glance

Dr. Fayçal Hamdi - For Course Use Only - 2026

Adversarial Search and Game Playing

Course Learning Outcome

CLO5: Evaluate distinct adversarial search strategies and its algorithms.

Lecture purpose

- Represent competitive decision problems as **game trees**.
- Explain how **minimax** computes optimal play under perfect information.
- Use **alpha-beta pruning** to obtain the same minimax decision with fewer explored nodes.
- Introduce practical game-playing methods based on **cutoff search**, **evaluation functions**, and **Monte Carlo Tree Search**.

Conceptual structure

Stage	Core question
Game formulation	How do we model a competitive environment?
Optimal play	How does minimax reason about an opponent?
Pruning	How can alpha-beta avoid useless branches?
Approximation	What if the game tree is too large to search completely?

💡 Main idea: adversarial search chooses actions by reasoning about what another rational agent may do in response.

Learning Objectives

Dr. Fayçal Hamdi - For Course Use Only - 2026

By the end of Lecture 5, you will be able to:

1. Formulate a two-player deterministic game using states, actions, terminal tests, and utility functions.
2. Explain the difference between single-agent search and adversarial search.
3. Construct and interpret a **game tree** with MAX and MIN levels.
4. Compute minimax values by backing up utilities from terminal states.
5. Apply **alpha-beta pruning** and explain why it preserves the minimax decision.
6. Use cutoff depth and evaluation functions for practical game-tree search.
7. Compare minimax, alpha-beta search, heuristic game search, stochastic-game search, and Monte Carlo Tree Search.

Lecture Roadmap

Dr. Fayçal Hamdi - For Course Use Only - 2026

From game formulation to efficient strategic search

Part	Main idea	Student output
1. Game formulation	Competitive environments require modeling how agents' actions interact.	Define players, legal actions, terminal states, and utilities.
2. Minimax reasoning	MAX chooses a move assuming MIN will respond optimally.	Back up values in a game tree and select the best move.
3. Alpha-beta pruning	Some branches cannot influence the final decision.	Identify alpha and beta cutoffs on small trees.
4. Practical game AI	Real games require approximations and sampling.	Explain cutoff search, evaluation functions, and MCTS.

Part I

Dr. Fayçal Hamdi — Course Use Only - 2026

Game Formulation and Strategic Reasoning

In adversarial search, the environment contains another decision maker whose actions are intentionally opposed to ours.

Dr. Fayçal Hamdi — For Course Use Only - 2026

Why Adversarial Search?

Competitive problems are not ordinary path-finding problems

Chess

Each move changes the position, but the opponent actively chooses replies that make winning harder.

Tic-tac-toe

The game tree is small enough to reason about perfect play and terminal utilities.

Checkers / Go

The search space is enormous, so practical systems rely on pruning, evaluation, and sampling.

Security planning

Defensive choices must consider possible attacks from an intelligent adversary.

💡 The solution is not simply a path to a goal. It is a **strategy**: a choice of action for each possible situation that may arise.

Single-Agent Search versus Adversarial Search

The source of uncertainty is different

Aspect	Single-agent search	Adversarial search
Environment	Usually modeled as passive	Contains another decision maker
Objective	Reach a goal state or minimize path cost	Maximize utility against an opponent
Search structure	State-space graph or search tree	Game tree with alternating players
Solution	Path or action sequence	Strategy or best move under assumptions
Main difficulty	Large state spaces	Large state spaces plus opponent responses

A move that looks good locally may be bad if it allows the opponent a strong reply.

Deterministic Two-Player Zero-Sum Games

A clean setting for adversarial search

The classical minimax model assumes:

- two players, usually called **MAX** and **MIN**;
- deterministic actions with fully known effects;
- perfect information: both players observe the full state;
- alternating turns;
- terminal utilities where one player's gain is the other player's loss.

In a zero-sum game, if MAX receives utility u , then MIN receives utility $-u$.

This model is simple, but it is powerful enough to explain the core logic of game-playing AI.

Formal Game Model

Components of an adversarial search problem

$$G = (S, s_0, Players, A, Result, Terminal, Utility)$$

Component	Meaning	Example
S	Set of possible game states	Board configurations
s_0	Initial state	Starting board
$Players$	Function indicating whose turn it is	MAX or MIN
$A(s)$	Legal actions in state s	Legal moves
$Result(s, a)$	State after action a	New board position
$Terminal(s)$	Whether the game is over	Win, loss, draw
$Utility(s)$	Value of a terminal state for MAX	+1, 0, -1

Utility Functions

Dr. Fayçal Hamdi - For Course Use Only - 2026

Terminal states must be comparable

A utility function assigns numerical values to terminal states from the perspective of MAX:

$$Utility(s) = \begin{cases} +1 & \text{if MAX wins} \\ 0 & \text{if the game is a draw} \\ -1 & \text{if MAX loses} \end{cases}$$

For games with richer scoring, utilities may represent final score difference, probability of winning, or another performance measure.

💡 The exact numbers matter less than their ordering: MAX prefers larger utilities, and MIN prefers smaller utilities.

Game Trees

Dr. Fayçal Hamdi - Course Use Only - 2026

Alternating levels of MAX and MIN decisions

A **game tree** expands possible moves from the current state. Levels alternate between the players:

- MAX nodes represent choices for the agent we are controlling;
- MIN nodes represent possible replies by the opponent;
- leaf nodes represent terminal states or cutoff states;
- values are backed up from leaves toward the root.

Even simple games can produce enormous game trees because the branching factor is repeatedly multiplied by depth.

Branching Factor in Games

Game trees grow very quickly

If a game has average branching factor b and we search to depth m , the number of nodes may be on the order of

$$O(b^m)$$

For example, with $b = 30$ and depth $m = 4$:

$$30^4 = 810000$$

This is why game AI needs pruning, good move ordering, evaluation functions, and sometimes sampling.

Part II

Dr. Fayçal Hamdi - Course Use Only - 2026

Minimax Search

Minimax chooses the move that maximizes our guaranteed outcome, assuming the opponent acts optimally.

Dr. Fayçal Hamdi — For Course Use Only - 2026

Minimax Intuition

Dr. Fayçal Hamdi - For Course Use Only - 2026

Choose the best move after considering the opponent's best reply

MAX perspective

MAX wants the highest utility. At a MAX node, the backed-up value is the maximum of the successors:

$$V(s) = \max_a V(\text{Result}(s, a))$$

MIN perspective

MIN wants the lowest utility for MAX. At a MIN node, the backed-up value is the minimum of the successors:

$$V(s) = \min_a V(\text{Result}(s, a))$$

💡 Minimax does not assume the opponent makes mistakes. It plans for the strongest rational response.

Minimax Value

Recursive definition

For a deterministic two-player game:

$$Minimax(s) = \begin{cases} Utility(s) & \text{if } Terminal(s) \\ \max_{a \in A(s)} Minimax(Result(s, a)) & \text{if } Player(s) = MAX \\ \min_{a \in A(s)} Minimax(Result(s, a)) & \text{if } Player(s) = MIN \end{cases}$$

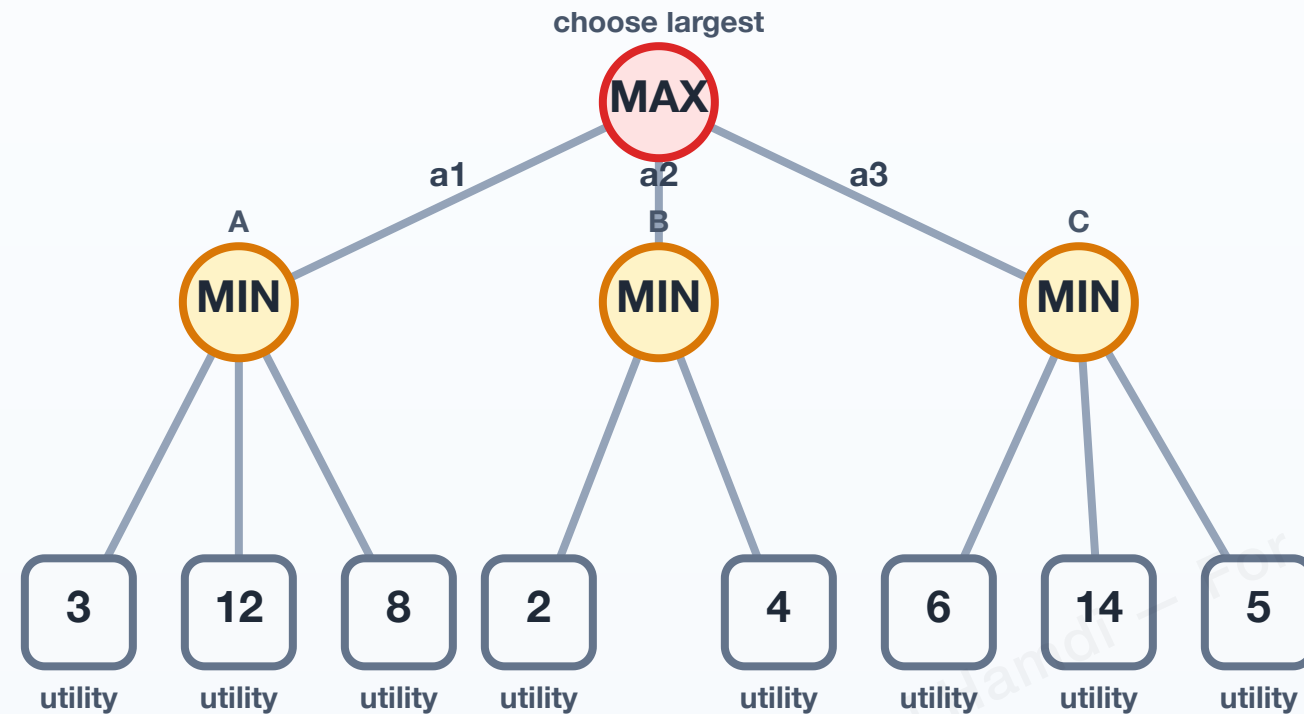
The best action for MAX at the root is:

$$a^* = \arg \max_{a \in A(s_0)} Minimax(Result(s_0, a))$$

Minimax Walkthrough

Dr. Fayçal Hamdi - Course Use Only - 2026

Backing up utilities from leaves to the root



Step 1 – start at MAX

What happens now?

MAX must choose an action, but each resulting state is controlled by MIN. Leaf values are utilities from MAX's perspective.

Possible actions

$a1 \rightarrow A$, $a2 \rightarrow B$, $a3 \rightarrow C$

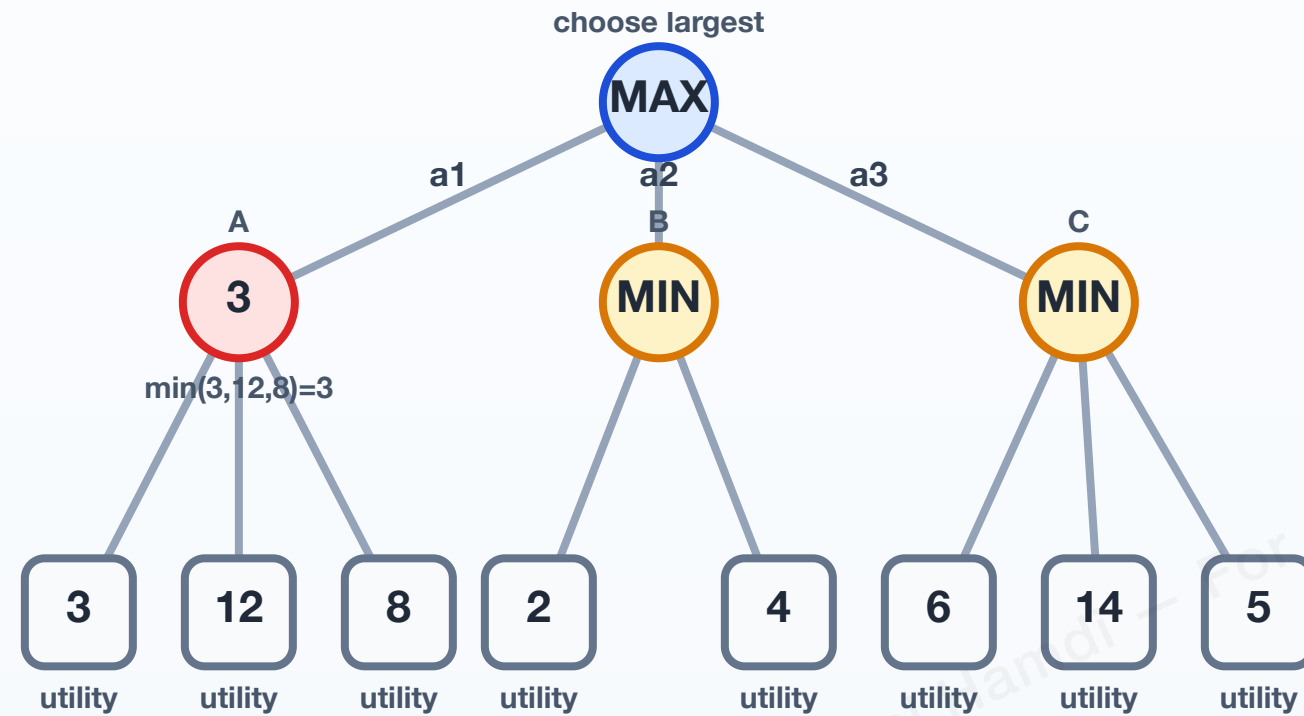
Question

Which action gives MAX the best guaranteed outcome?

Legend

MAX MIN Current Backed-up value

Backing up utilities from leaves to the root



Step 2 — back up node A

What happens now?

At A, MIN chooses the outcome that is worst for MAX. Therefore A receives the smallest leaf utility.

Calculation

$$\min(3, 12, 8) = 3$$

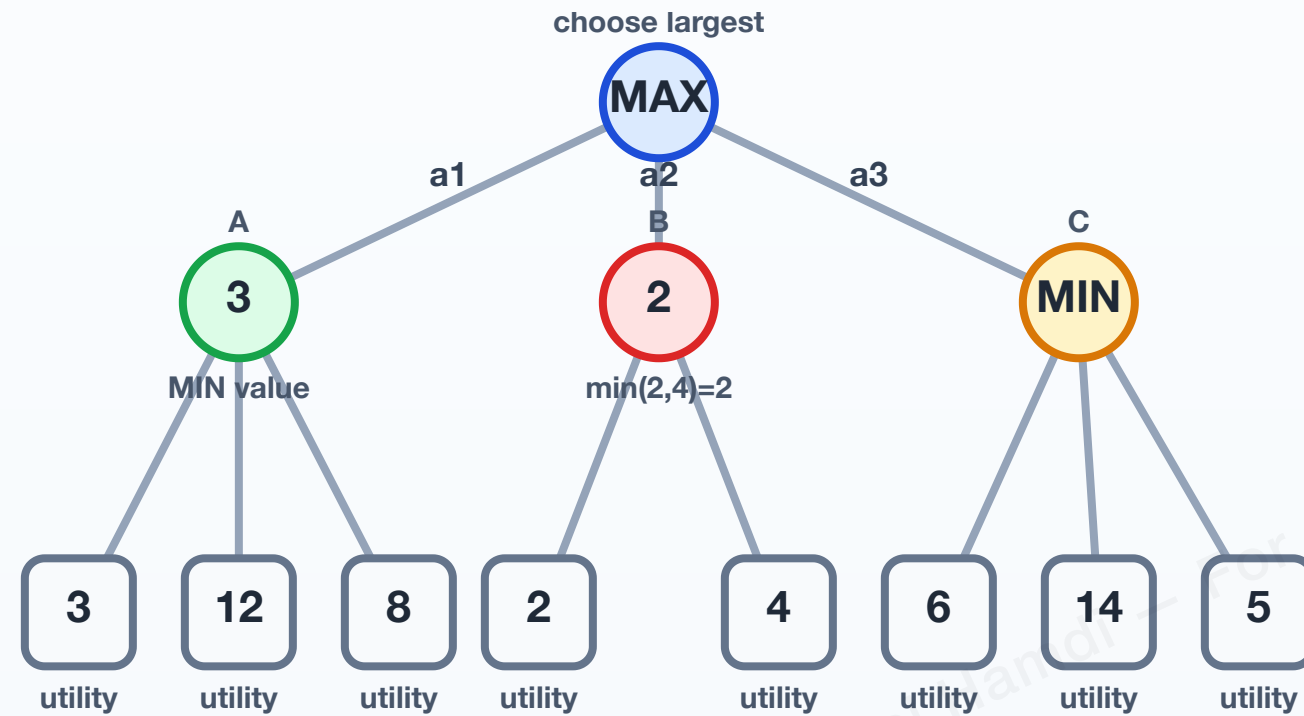
Backed-up value

$$A = 3$$

Legend

MAX MIN Current Backed-up value

Backing up utilities from leaves to the root



Step 3 — back up node B

What happens now?

At B, MIN compares 2 and 4 and chooses the smaller value. That value is backed up toward MAX.

Calculation

$$\min(2, 4) = 2$$

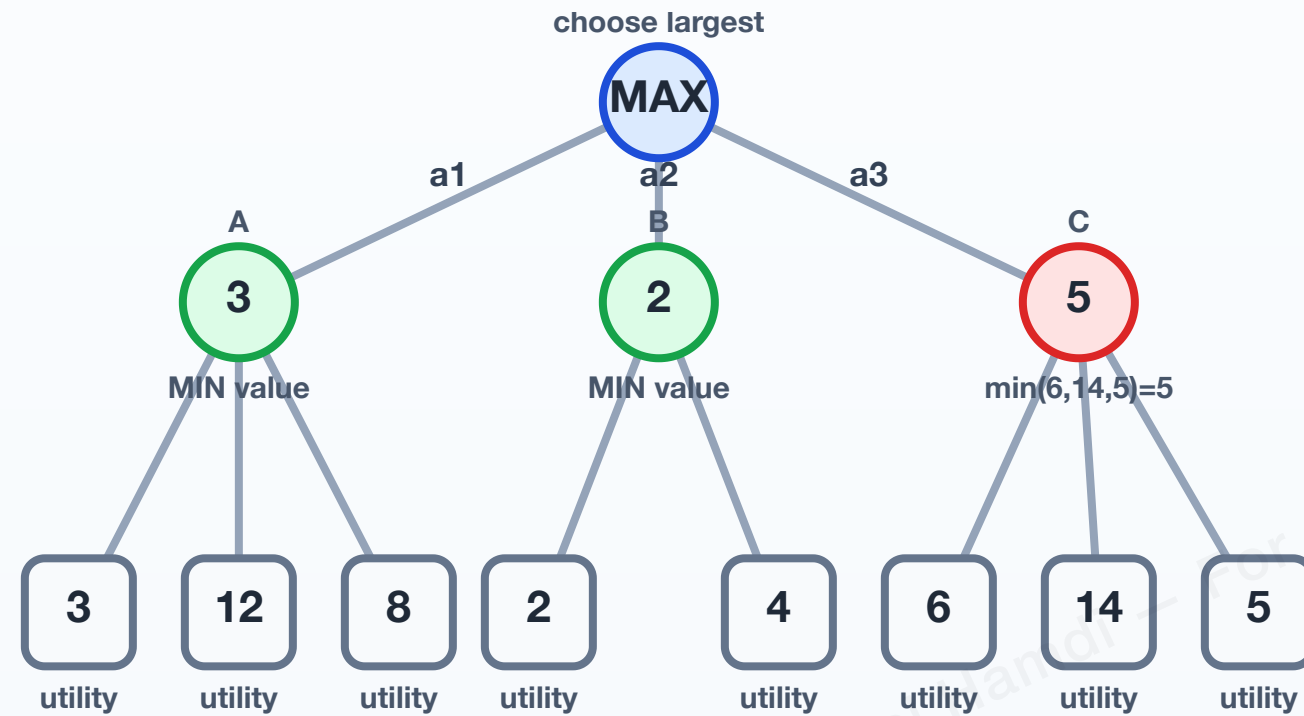
Backed-up values

A = 3, B = 2

Legend

MAX MIN Current Backed-up value

Backing up utilities from leaves to the root



Step 4 — back up node C

What happens now?

At C, MIN chooses the smallest utility among 6, 14, and 5. Therefore C has minimax value 5.

Calculation

$$\min(6, 14, 5) = 5$$

Backed-up values

$$A = 3, B = 2, C = 5$$

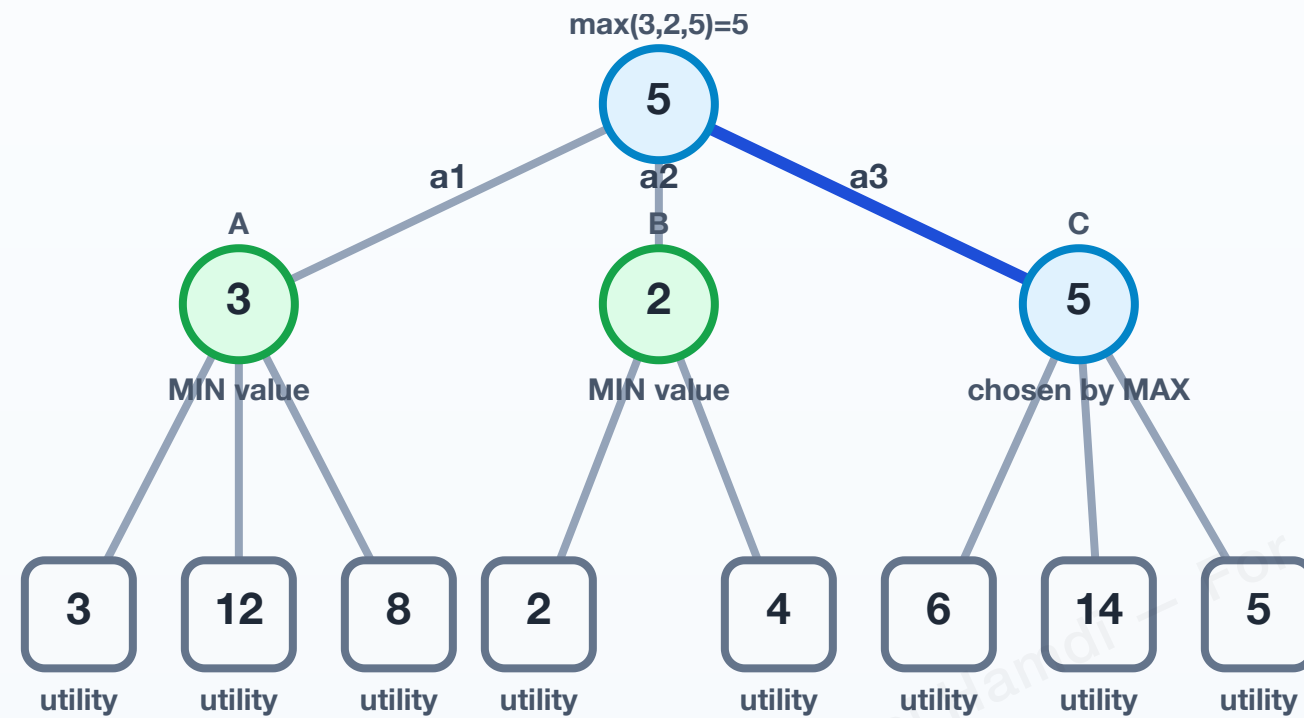
Next

MAX can now compare all three actions.

Legend

MAX MIN Current Backed-up value

Backing up utilities from leaves to the root



Step 5 – MAX chooses the largest value

What happens now?

MAX compares the backed-up values 3, 2, and 5. Action a3 gives the highest guaranteed utility, so MAX chooses it.

Calculation

$$\max(3, 2, 5) = 5$$

Decision

Choose action a3 → C

Root minimax value

5

Legend

MAX MIN Current Backed-up value

Minimax Algorithm — Pseudocode

Dr. Fayçal Hamdi - Course Use Only - 2026

Depth-first recursive game-tree search

```
def minimax_decision(state):  
    best_action = None  
    best_value = -infinity  
  
    for action in actions(state):  
        value = min_value(result(state, action))  
        if value > best_value:  
            best_value = value  
            best_action = action  
  
    return best_action  
  
def max_value(state):  
    if terminal_test(state):  
        return utility(state)  
    value = -infinity  
    for action in actions(state):  
        value = max(value, min_value(result(state, action)))
```

Minimax Properties

Dr. Fayçal Hamdi - For Course Use Only - 2026

Correct but potentially expensive

Property	Minimax behavior
Completeness	Complete if the game tree is finite.
Optimality	Optimal against an optimal opponent under the model assumptions.
Time complexity	$O(b^m)$ where b is branching factor and m is maximum search depth.
Space complexity	$O(bm)$ for depth-first implementation.
Main limitation	The full game tree is usually too large to search completely.

Minimax is a decision rule, not a guarantee of winning. If the opponent also plays optimally, the final result is determined by the game structure.

Minimax and Strategy

A move is chosen using a conditional view of the future

A single move at the root implicitly represents a larger strategy:

- MAX chooses an action now.
- MIN may choose several possible replies.
- MAX must be ready to respond to each reply.
- The backed-up value summarizes the outcome under optimal play from both sides.

💡 The minimax value of a node is the utility MAX can guarantee from that node if both players play optimally.

Part III

Dr. Fayçal Hamdi - Course Use Only - 2026

Alpha-Beta Pruning

Alpha-beta pruning keeps the minimax decision but avoids evaluating branches that cannot affect it.

Dr. Fayçal Hamdi — For Course Use Only - 2026

Motivation for Alpha-Beta Pruning

Many branches are irrelevant to the final choice

Minimax

Explores **every branch** to the chosen depth.

Alpha-beta

Stops exploring when a branch **cannot affect the final choice**.

Pruning means stopping the exploration of a branch because its value cannot change the final decision.

Alpha-beta changes how much we search — not the final decision.

Alpha and Beta

Bounds on what each player can guarantee

Alpha α

The best value MAX can guarantee so far along the current path.

α = best lower bound for MAX

MAX tries to increase α .

Beta β

The best value MIN can guarantee so far along the current path.

β = best upper bound for MAX

MIN tries to decrease β .

A cutoff occurs when the current branch is provably unable to improve the decision already available to an ancestor.

Alpha-Beta Cutoff Rule

When can we stop searching a branch?

The central condition is:

$$\alpha \geq \beta$$

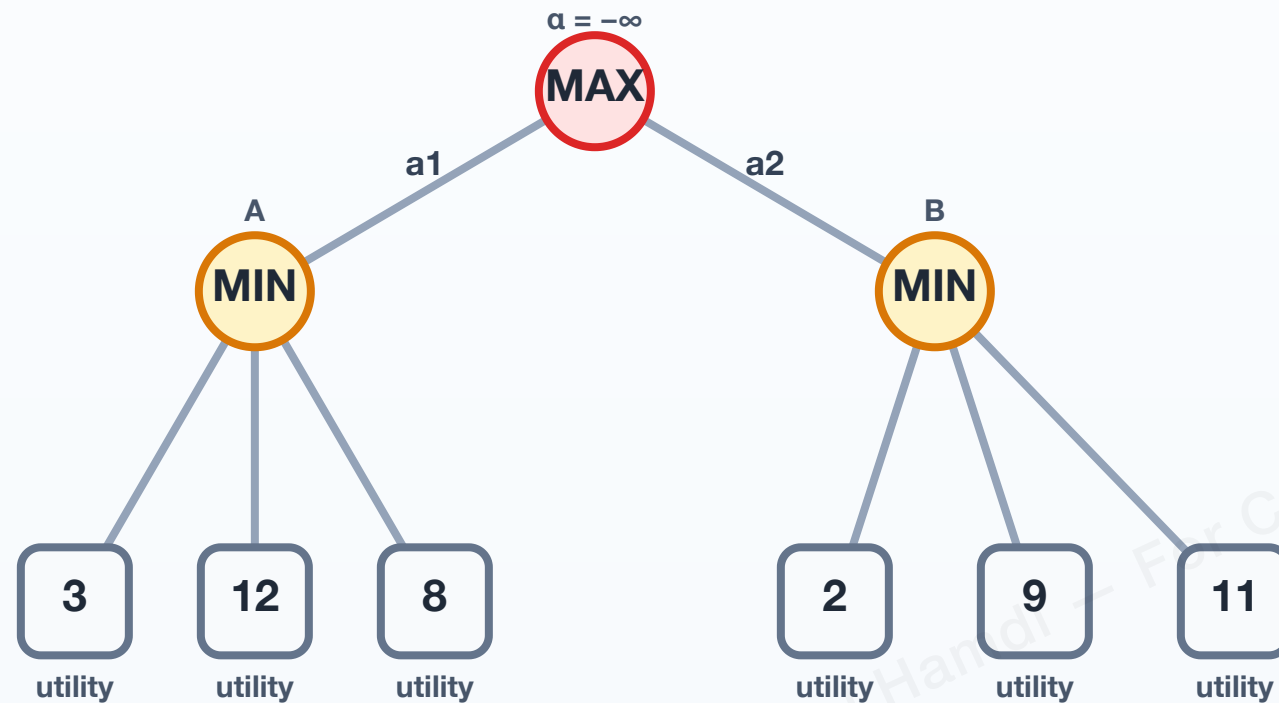
At that point, the current branch cannot influence the final decision under optimal play.

- At a MAX node, values worse than an existing MAX alternative are irrelevant.
- At a MIN node, values better for MAX may never be allowed by MIN.

💡 Alpha-beta pruning is safe because it removes only branches that cannot change the backed-up minimax value.

Alpha-Beta Walkthrough

Pruning branches that cannot affect the decision



Step 1 — begin with bounds

What happens now?

Alpha-beta computes the same decision as minimax while maintaining bounds. α is a lower bound on what MAX can guarantee; β is an upper bound on what MIN can allow.

Initial bounds

$\alpha = -\infty$ $\beta = +\infty$

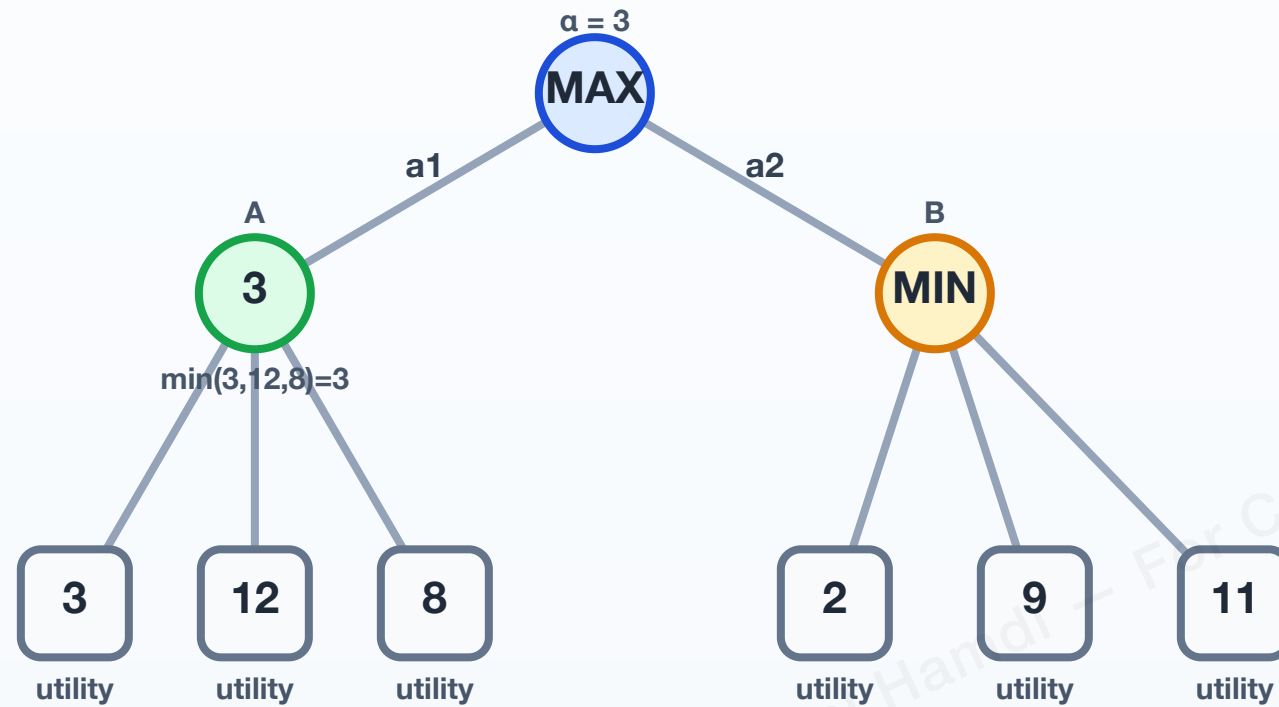
Search order

Evaluate A first, then B.

Legend

MAX MIN Current Evaluated Pruned

Pruning branches that cannot affect the decision



Step 2 — evaluate A

What happens now?

MIN chooses the smallest utility under A. Thus A has value 3, giving MAX a guaranteed value of at least 3.

A

$$\min(3, 12, 8) = 3$$

Update at root

$$\alpha = \max(-\infty, 3) = 3$$

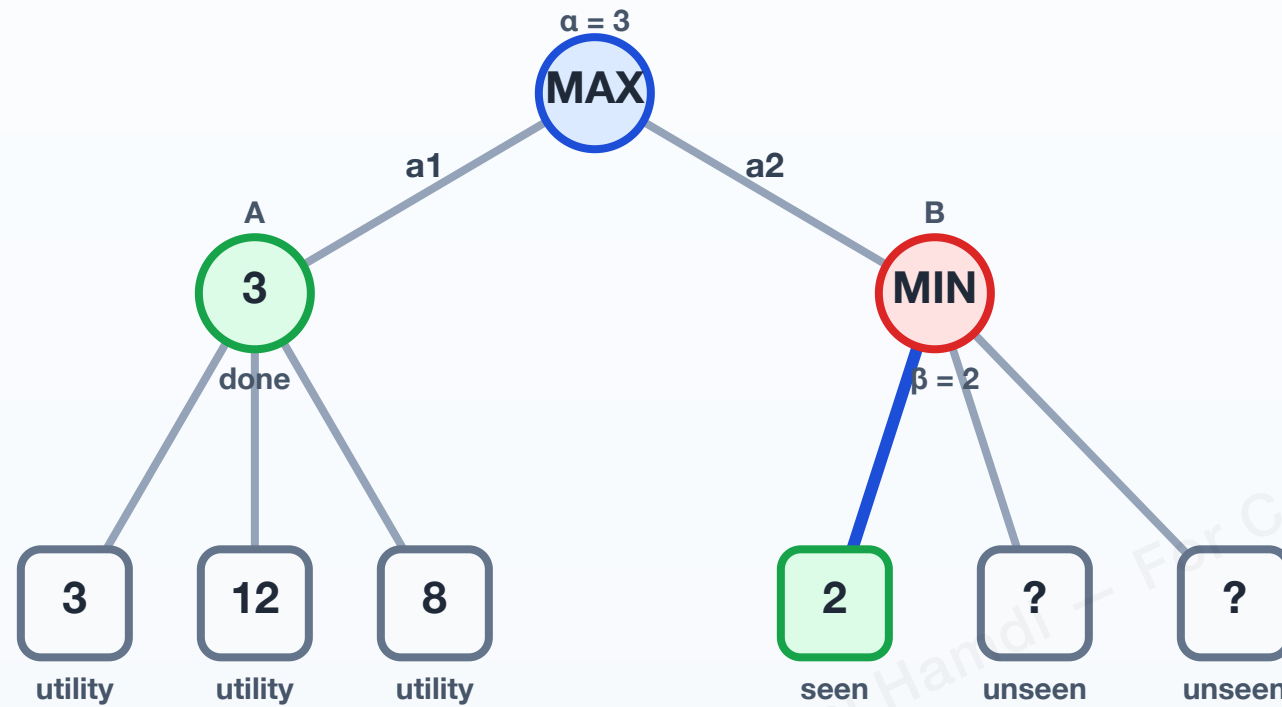
Meaning

MAX already has an action worth 3.

Legend

MAX MIN Current Evaluated Pruned

Pruning branches that cannot affect the decision



Step 3 — inspect B's first child

What happens now?

At B, MIN sees a value of 2, so β becomes 2. But MAX already has $\alpha = 3$ from action a1.

At B

$$\beta = \min(+\infty, 2) = 2$$

Cutoff test

$$\beta = 2 \quad \alpha = 3 \quad \beta \leq \alpha$$

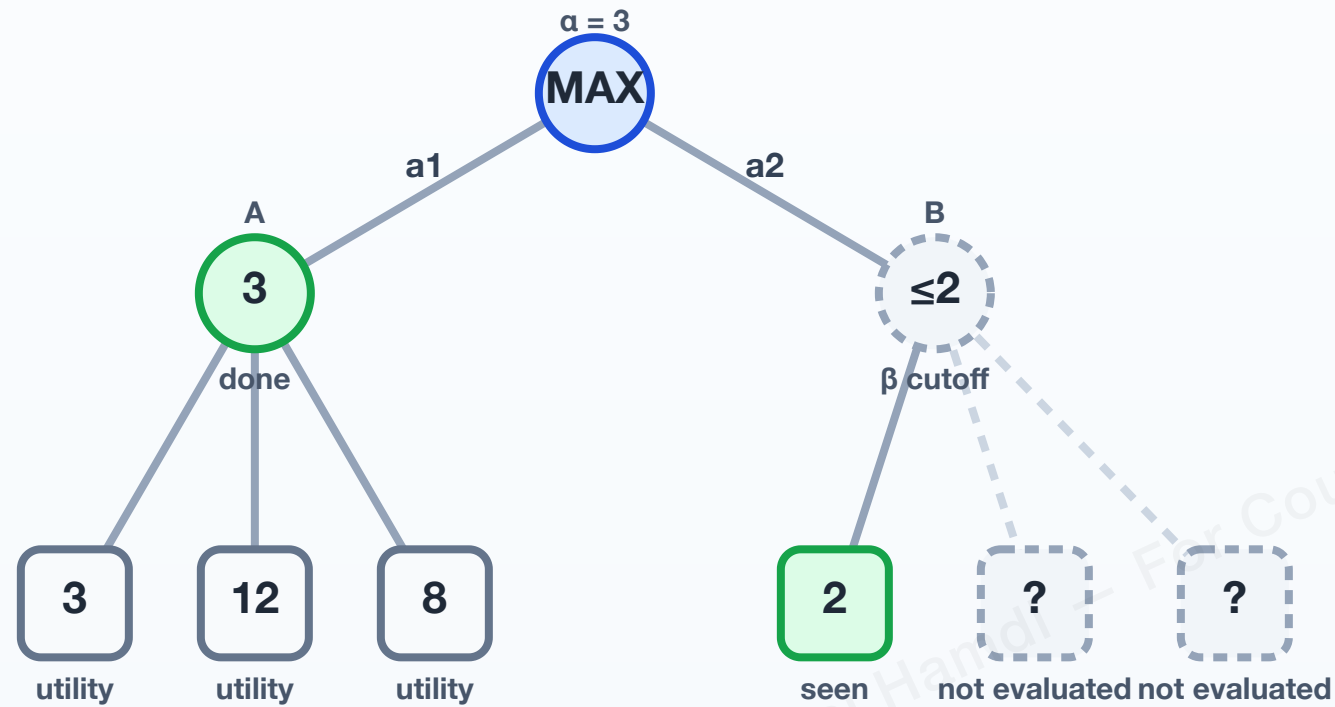
Consequence

B cannot improve MAX's current choice.

Legend

MAX MIN Current Evaluated Pruned

Pruning branches that cannot affect the decision



Step 4 — prune the remaining children

What happens now?

Because $\beta \leq \alpha$, the remaining children of B cannot affect MAX's decision. Alpha-beta stops exploring this branch.

Cutoff

$$2 \leq 3$$

Pruned

B's two remaining children are not evaluated.

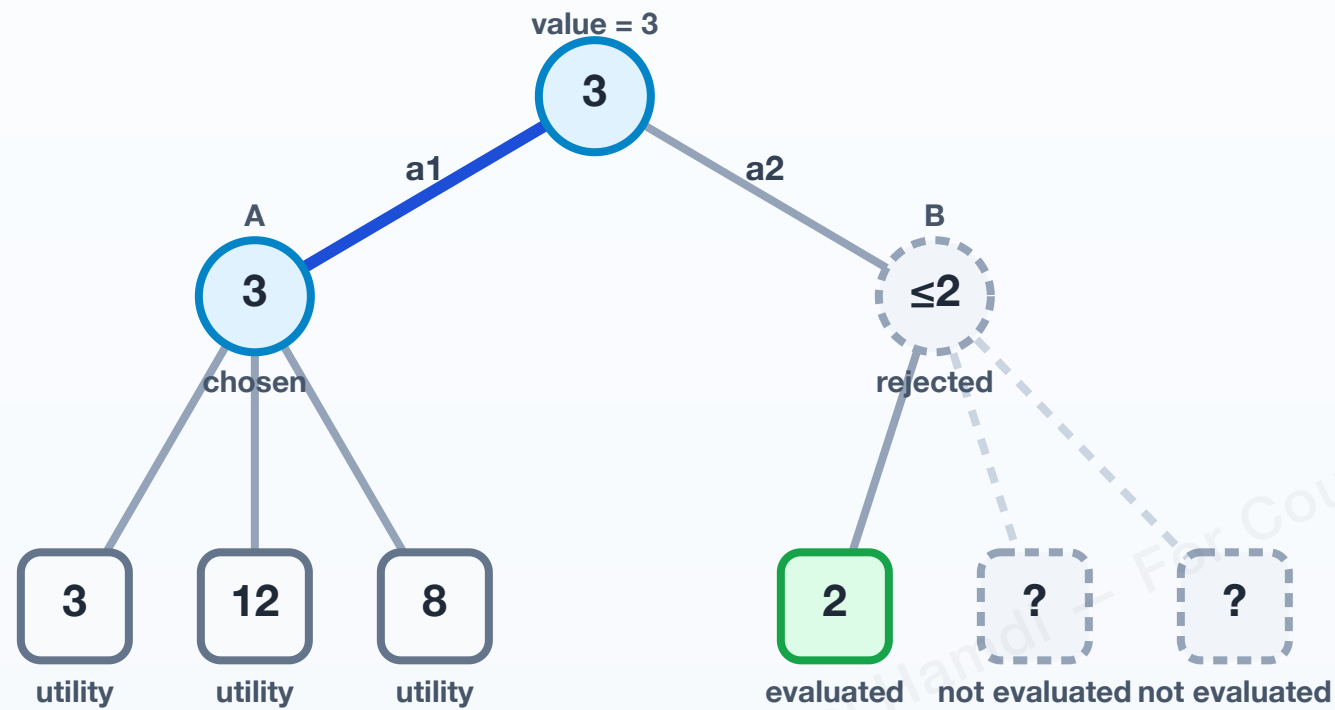
Key idea

Their utilities are unnecessary for the decision.

Legend

MAX MIN Current Evaluated Pruned

Pruning branches that cannot affect the decision



Step 5 — same decision, less search

What happens now?

MAX chooses action a1 with value 3. Alpha-beta gives the same minimax decision without evaluating every leaf.

Decision

Choose action a1 → A

Root value

3

Saving

2 of the 6 leaves were never evaluated.

Legend

MAX MIN Current Evaluated Pruned

Alpha-Beta Algorithm — Pseudocode

Minimax with bounds

```
def alpha_beta_search(state):
    value, action = max_value(state, -infinity, infinity)
    return action

def max_value(state, alpha, beta):
    if terminal_test(state):
        return utility(state), None

    value = -infinity
    best_action = None
    for action in actions(state):
        child_value, _ = min_value(result(state, action), alpha, beta)
        if child_value > value:
            value = child_value
            best_action = action
        if value >= beta:
            return value, best_action
    alpha = max(alpha, value)
```

Alpha-Beta MIN Procedure

The symmetric part of the algorithm

```
def min_value(state, alpha, beta):
    if terminal_test(state):
        return utility(state), None

    value = infinity
    best_action = None
    for action in actions(state):
        child_value, _ = max_value(result(state, action), alpha, beta)
        if child_value < value:
            value = child_value
            best_action = action
        if value <= alpha:
            return value, best_action
        beta = min(beta, value)
    return value, best_action
```

💡 The algorithm still performs depth-first search, but it carries α and β values down the tree.

Effect of Move Ordering

Pruning is strongest when good moves are examined early

Alpha-beta pruning always returns the same minimax decision, but the amount of pruning depends strongly on move ordering.

Move ordering	Practical effect
Best moves first	Many cutoffs; deeper search becomes possible.
Random order	Some pruning, but less predictable.
Worst moves first	Little pruning; behavior approaches plain minimax.

With perfect move ordering, alpha-beta can reduce the effective complexity from about $O(b^m)$ to about $O(b^{m/2})$.

Practical Move Ordering Ideas

How can an algorithm guess good moves first?

Captures and threats

In board games, moves that capture pieces or create immediate threats are often searched early.

Previous best move

Iterative deepening can reuse the best move found at a shallower depth.

Domain heuristics

Game-specific knowledge ranks moves before exact search begins.

Transposition tables

Previously evaluated positions can guide ordering and avoid repeated work.

Part IV

Dr. Fayçal Hamdi - Course Use Only - 2026

Practical Game-Playing Search

Real games are often too large for complete minimax search, so practical agents combine search with evaluation and sampling.

Dr. Fayçal Hamdi — For Course Use Only - 2026

Cutoff Search

Dr. Fayçal Hamdi - For Course Use Only - 2026

Stop before terminal states

In many games, searching all the way to terminal states is impossible. Instead, the algorithm searches only to a fixed depth or until a time limit is reached.

At a cutoff state, the terminal utility is replaced by an evaluation function:

$$V(s) \approx Eval(s)$$

An **evaluation function** estimates how good a nonterminal state is for MAX.

Cutoff search turns exact game-tree reasoning into a controlled approximation.

Evaluation Functions

Convert board features into an estimated value

A common evaluation function is a weighted sum of features:

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_k f_k(s)$$

where features may describe material advantage, mobility, king safety, center control, threats, or other domain-specific patterns.

💡 The evaluation function should be fast enough to apply many times during search, but accurate enough to rank states meaningfully.

Example Evaluation Function

A simplified chess-like scoring model

Feature	Meaning	Weight	Example value	Contribution
$f_1(s)$	Material advantage	9	+1	+9
$f_2(s)$	Mobility advantage	2	+3	+6
$f_3(s)$	King safety risk	-5	1	-5
$f_4(s)$	Center control	1	+4	+4
Total				14

The positive score suggests that the position is favorable for MAX, but it is still an estimate rather than a guaranteed outcome.

Horizon Effect

Shallow search can miss delayed consequences

The **horizon effect** occurs when a serious consequence lies just beyond the search depth.

Example: a move appears good at depth 4, but at depth 5 the opponent has a decisive tactical reply.

Cutoff search may overvalue states that postpone losses beyond the search horizon.

A common mitigation is **quiescence search**, which extends search in unstable positions such as captures, checks, or immediate threats.

Stochastic Games

Dr. Fayçal Hamdi - Course Use Only - 2026

Some games include chance events

In games with randomness, game trees may include **chance nodes** in addition to MAX and MIN nodes.

For a chance node, the value is an expected value:

$$V(s) = \sum_i P(s_i \mid s) V(s_i)$$

This leads to algorithms such as **expectiminimax**.

Backgammon and many card or dice games require reasoning about both opponent choices and random outcomes.

Monte Carlo Tree Search

Use sampling instead of exhaustive tree expansion

Monte Carlo Tree Search builds a partial game tree guided by repeated simulations.

A typical MCTS iteration has four phases:

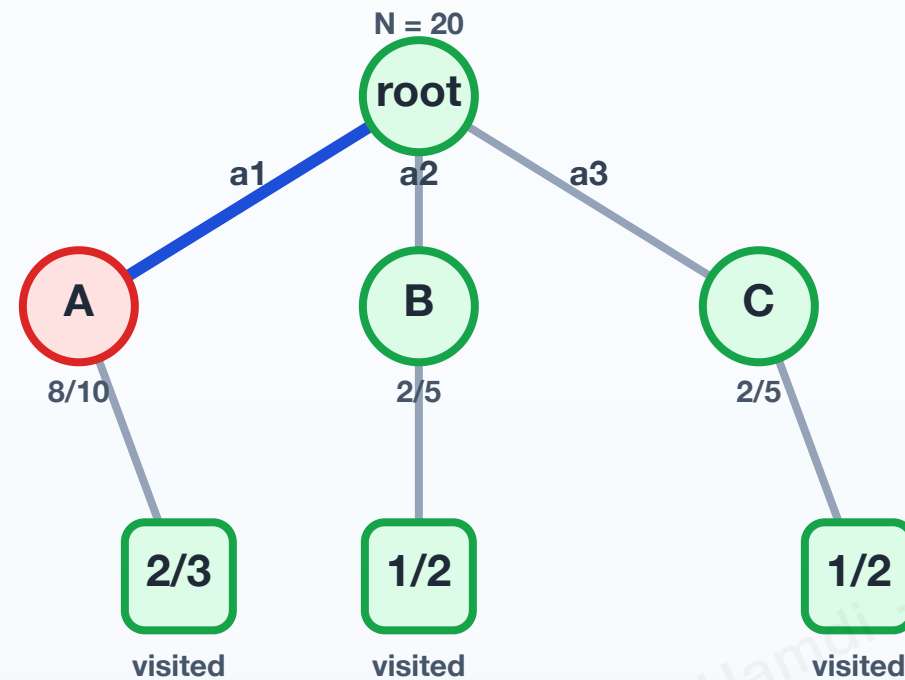
1. **Selection:** choose a path through the current tree.
2. **Expansion:** add a new child node.
3. **Simulation:** play out a fast rollout.
4. **Backpropagation:** update statistics on the path.

💡 MCTS is especially useful when evaluation functions are difficult to design or the branching factor is very large.

MCTS Walkthrough

Dr. Fayçal Hamdi - Course Use Only - 2026

Selection, expansion, simulation, and backpropagation



Step 1 — selection

What happens now?

Starting at the root, MCTS follows the tree policy, balancing promising outcomes with further exploration. In this snapshot, the policy selects A.

Statistics

W/N = wins for the root player / visits

Tree policy

A is selected by the exploration–exploitation rule.

Next

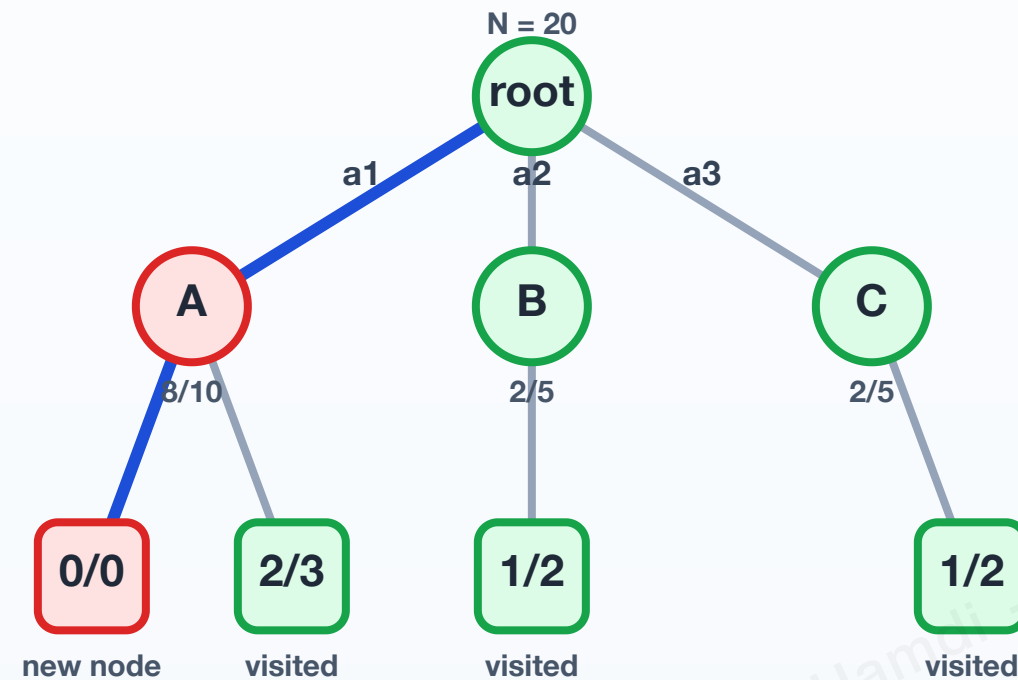
A still has an untried action, so it can be expanded.

Legend

Selected path

Visited node

Selection, expansion, simulation, and backpropagation



Step 2 – expansion

What happens now?

A has an unexplored action. MCTS adds one previously unvisited child to the search tree.

New child

A1 is added with $W/N = 0/0$.

Important

Expansion adds one new search-tree node.

Next

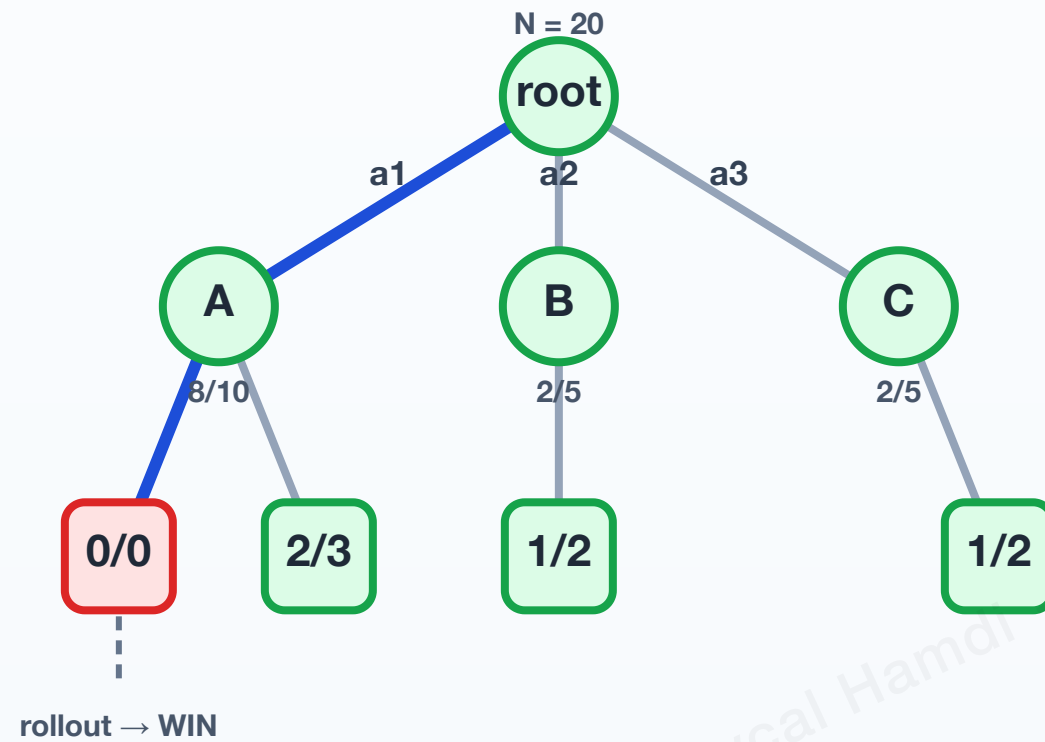
Estimate the outcome from A1 by simulation.

Legend

Selected / new

Visited node

Selection, expansion, simulation, and backpropagation



Step 3 – simulation

What happens now?

From the newly expanded state A1, perform a rollout until a terminal state is reached.

Rollout

Follow a fast simulation policy beyond the current search tree.

Terminal result

Win for the root player.

Reward

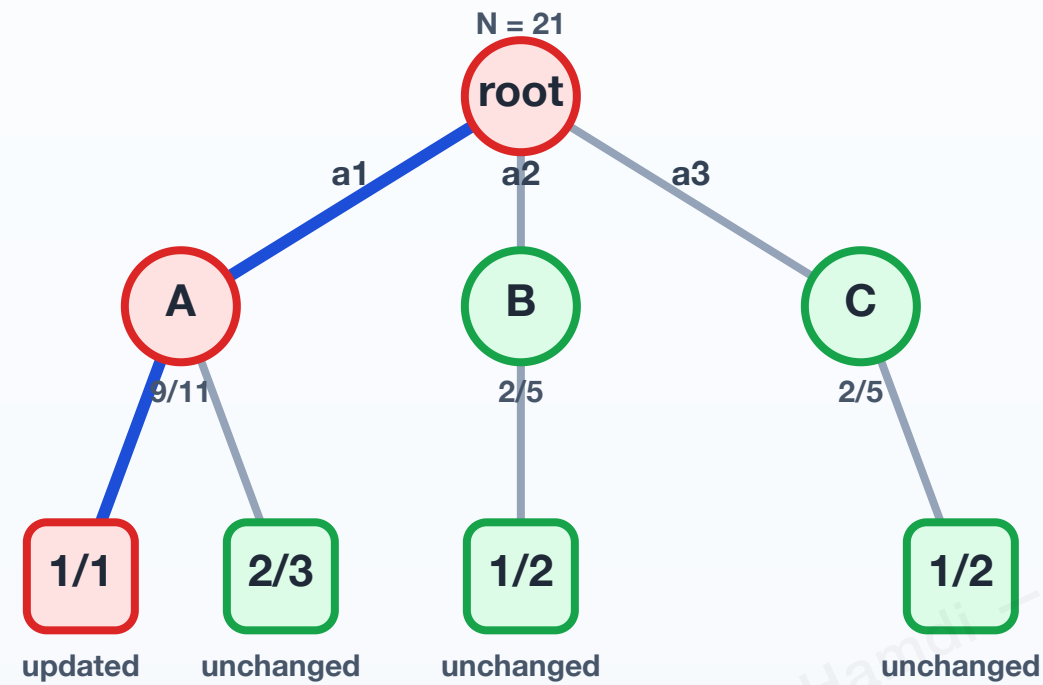
Record reward = 1.

Legend

Simulation start

Visited node

Selection, expansion, simulation, and backpropagation



Step 4 – backpropagation

What happens now?

Propagate the rollout result back along the selected path, updating visits and wins for every node on that path.

A1

0/0 → 1/1

A

8/10 → 9/11

Root

N = 20 → 21

Next

Repeat selection → expansion → simulation → backpropagation many times.

Legend

Updated path

Unchanged node

UCT Selection Rule

Balancing exploitation and exploration

A common selection rule in MCTS is UCT:

$$UCT(s_i) = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N}{n_i}}$$

where:

Symbol	Meaning
w_i	Number or sum of rewards obtained through child i
n_i	Number of visits to child i
N	Number of visits to the parent node
c	Exploration constant

The first term prefers moves with high average reward. The second term encourages trying less-visited moves.

Comparing Game-Search Strategies

Different assumptions lead to different algorithms

Strategy	Main idea	Strength	Limitation
Minimax	Search full deterministic game tree	Optimal under ideal assumptions	Exponential growth
Alpha-beta	Prune branches that cannot affect minimax	Same result as minimax with less work	Depends on move ordering
Cutoff + Eval	Search to limited depth and estimate leaves	Practical for large games	Quality depends on evaluation function
Expectiminimax	Add chance nodes and expected values	Handles stochastic outcomes	Even larger trees
MCTS	Use simulations to guide partial tree growth	Effective in huge games	Requires many simulations and careful tuning

Choosing an Adversarial Search Method

A practical decision guide

Situation	Suitable method	Reason
Small deterministic game tree	Minimax	Exact optimal play is feasible.
Deterministic game with large tree	Alpha-beta with move ordering	Same minimax value with fewer explored nodes.
Game too large for terminal search	Cutoff search + evaluation	Provides approximate decisions under resource limits.
Game includes chance events	Expectiminimax	Incorporates expected values.
Evaluation function is weak or hard to design	MCTS	Learns from sampled outcomes.
Strict time limit	Iterative deepening + alpha-beta	Always has a best-so-far move available.

Worked Trace Activity

Dr. Fayçal Hamdi - For Course Use Only - 2026

Compute minimax and identify pruning opportunities

Consider a root MAX node with three MIN children:

```
A: [4, 7, 2]  
B: [6, 5, 9]  
C: [3, 8, 1]
```

Tasks:

1. Compute the minimax value of each MIN node.
2. Compute the minimax value of the root.
3. Assume left-to-right evaluation. Which branches could alpha-beta prune after a good first move is already known?
4. How would the answer change if the children were ordered differently?

Expected insight: minimax values are obtained by backing up minimum values at MIN nodes and maximum values at MAX nodes. Alpha-beta pruning depends heavily on the order in which branches are searched.

Mini-Quiz

Dr. Fayçal Hamdi - For Course Use Only - 2026

Check your understanding

1. What is the difference between a terminal utility and an evaluation function?
2. Why does a MIN node take the minimum of its successor values?
3. Why does alpha-beta pruning return the same move as minimax?
4. What does the condition $\alpha \geq \beta$ mean intuitively?
5. Why can poor move ordering reduce the benefit of alpha-beta pruning?
6. Why is MCTS useful when the game tree is extremely large?

Core answer: adversarial search evaluates actions by anticipating rational responses from other agents.

Lecture 5 Summary

Dr. Fayçal Hamdi - For Course Use Only - 2026

Key ideas to retain

Game-tree reasoning

- Game states are evaluated from the perspective of MAX.
- MAX chooses high values; MIN chooses low values.
- Minimax backs utilities up from leaves to the root.
- The chosen action is optimal under the model assumptions.

Practical efficiency

- Alpha-beta prunes branches that cannot affect the decision.
- Good move ordering greatly improves pruning.
- Cutoff search and evaluation functions make game search practical.
- MCTS uses simulations to guide partial tree expansion.

End-of-Lecture Check

Five questions to test your understanding

1. What components are needed to formulate an adversarial search problem?
2. Why is a strategy richer than a single path in a game tree?
3. How are values propagated at MAX and MIN nodes?
4. What information do α and β represent?
5. When would you prefer MCTS over alpha-beta search with a handcrafted evaluation function?

Preparation for next lecture: Review the difference between symbolic AI methods and data-driven learning. Lecture 6 surveys advanced AI subdomains such as machine learning, natural language processing, and computer vision.

Suggested Reading

Primary textbook

S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, Prentice Hall, 2020.

- Chapter 5: Adversarial Search and Games
- Focus especially on minimax, alpha-beta pruning, evaluation functions, stochastic games, and Monte Carlo Tree Search.

💡 When reading, pay close attention to the distinction between exact optimal search and resource-bounded approximate game playing.